

ADIVINA QUIÉN

Documentación del TPO

Programación III — UADE

Alumna: Baptista Candela Legajo: 1158810

Cómo encaré el problema

Lo primero que hice fue darme cuenta de que el juego que pide el enunciado es el mismo problema que el ejemplo del “número secreto” que vimos en clase para introducir Divide & Conquer. En ese ejemplo la computadora tiene que adivinar un número entre 1 y 100 y el jugador solo le dice “es mayor” o “es menor”. Acá, en vez de un rango de números, tengo un conjunto de 23 personajes, y en vez de “mayor o menor” tengo respuestas de sí o no. Pero la idea es idéntica: un espacio de candidatos que se achica con cada respuesta.

La segunda cosa que entendí, y que me parece la más importante de todo el trabajo, es que en cada turno **pasan dos cosas distintas** que se resuelven con algoritmos diferentes:

- 1. Elegir qué preguntar.** Antes de preguntar hay que decidir cuál de las características conviene. Eso lo resuelvo con Greedy.
- 2. Achicar la lista con la respuesta.** Una vez que me contestaron, tiro a la basura los personajes que ya no pueden ser. Eso es Divide & Conquer.

Al principio los tenía mezclados en la cabeza y no podía explicar dónde estaba cada patrón. Separarlos fue lo que me ordenó todo el proyecto. No son alternativas entre sí: greedy elige la pregunta, D&C aprovecha la respuesta. Por eso conviven en el mismo turno.

Divide & Conquer: cómo se achica la lista

El esquema que vimos en clase es este:

```
Algoritmo D&C(x)
  if CasoBase(x) return SoluciónDirecta(x)
  else
    descomponer(x)
    resolver cada subproblema
    combinar las soluciones
```

Lo traduje así a mi juego, y usé esos mismos nombres en el código para que se vea la correspondencia:

El esquema dice	En mi juego es	Método en el código
x	los personajes que todavía pueden ser el secreto	campo candidatos
CasoBase(x)	queda uno solo	esCasoBase()
SoluciónDirecta(x)	ese es el personaje, lanzo la suposición	solucionDirecta()
descomponer(x)	separar en “cumple el filtro” y “no cumple”	descomponer()
combinar	quedarme solo con el grupo que corresponde	recibirRespuesta()

Todo eso está en la clase Jugador. La puse ahí, y no dentro de la máquina, porque **el tablero del jugador humano se achica exactamente igual**: cuando uno pregunta “¿es hombre?” y le dicen que no, el programa descarta solas a las 11 mujeres. Es el mismo algoritmo para los dos; lo único que cambia es quién elige la pregunta.

Una diferencia con el esquema genérico

En el esquema de la cátedra se resuelven **todos** los subproblemas y después se combinan. En mi juego resuelvo **uno solo**: la respuesta del rival me dice en cuál de los dos grupos está el personaje, así que el otro lo descarto entero sin mirarlo.

Es la misma simplificación que hace la búsqueda binaria, y es justamente lo que hace que el algoritmo sirva. Si tuviera que revisar los dos grupos, no ganaría nada respecto de ir probando personaje por personaje.

Dónde está la recursión

La recursión **no está adentro de un método que se llama a sí mismo**: está repartida a lo largo de los turnos. Cada turno es el mismo problema que el anterior pero con la mitad de los candidatos.

Esta es la traza real de una partida que imprime el programa (el personaje secreto era Hugo):

```
descomponer(23) con "¿Es un hombre?" -> [12 | 11] respuesta SI -> quedan 12
descomponer(12) con "¿Es calvo?"      -> [ 6 |  6] respuesta SI -> quedan 6
descomponer(6)  con "¿Usa lentes?"    -> [ 3 |  3] respuesta NO -> quedan 3
descomponer(3)  con "¿Pelo colorado?" -> [ 1 |  2] respuesta NO -> quedan 2
descomponer(2)  con "¿Pelo negro?"    -> [ 1 |  1] respuesta SI -> queda 1
CasoBase -> SoluciónDirecta = Hugo
```

23 → 12 → 6 → 3 → 2 → 1. Es la misma forma que el ejemplo de clase, que iba 100 → 50 → 25 → 12 → 6 → 3 → 1.

Escribí también la clase BuscadorRecursivo, que es **el mismo algoritmo pero con la recursión escrita de forma literal**: el método resolver() se llama a sí mismo. Ahí se ve la correspondencia línea por línea con el esquema de la cátedra.

¿Por qué tengo las dos versiones? Porque la interfaz gráfica funciona por eventos: cuando el usuario tiene que hacer clic en un botón, no puedo dejar una llamada recursiva colgada esperando. Entonces la versión por turnos es la que usa Swing, y la recursiva es la que uso en el modo máquina contra máquina, donde el rival contesta al instante. **Son el mismo algoritmo, no dos algoritmos distintos.**

Greedy: cómo elijo qué preguntar

Acá aplico los cinco elementos que vimos en clase:

Elemento	En mi juego
Conjunto de candidatos	los filtros que todavía no pregunté
Función de selección	el filtro que deja los dos grupos más parejos
Función de factibilidad	que no lo haya usado y que realmente separe en dos grupos
Función de solución	que quede un solo personaje
Objetivo	usar la menor cantidad de preguntas

Por qué elijo por el peor caso

La máquina **no sabe qué le van a contestar**. Entonces, ¿con qué criterio compara dos preguntas? Miremos la tabla que imprime el programa en el primer turno, con los 23 personajes:

Pregunta	Cómo parte el grupo	Peor caso
¿Es un hombre?	12 / 11	12
¿Es calvo?	11 / 12	12
¿Usa lentes?	11 / 12	12
¿Tiene el pelo colorado?	8 / 15	15
¿Tiene el pelo negro?	8 / 15	15
¿Tiene el pelo amarillo?	7 / 16	16

Si eligiera pensando en que me va a ir bien, preguntaría “¿pelo amarillo?” con la ilusión de que me digan que sí y quedarme con 7. Pero si me dicen que no, me quedo con 16, o sea peor que antes de preguntar.

Como no controlo la respuesta, lo único que puedo controlar es **qué tan mal me puede ir**. Por eso elijo el filtro cuyo peor caso sea el más chico. Ese criterio se llama minimax: minimizo el máximo.

Y es greedy de verdad porque **decide mirando solo el turno actual**. No simula qué va a pasar después ni se arrepiente de preguntas ya hechas. Es “corto de vista” a propósito, igual que el algoritmo del cambio de monedas que vimos en clase.

La limitación de greedy, que no la escondó

En clase quedó claro que un algoritmo voraz **no es automáticamente correcto**: hay que demostrarlo. El ejemplo del apunte es el sistema de monedas británico anterior a 1971, donde greedy da 4 monedas cuando el óptimo son 3.

En mi caso puedo argumentar que sí llega al óptimo, con este razonamiento:

Cada pregunta de sí/no da como máximo 1 bit de información. Para distinguir entre 23 personajes necesito al menos $\lceil \log_2 23 \rceil = 5$ preguntas, haga lo que haga. Mi greedy resuelve en 5 preguntas en el peor caso. Como el mínimo teórico y mi resultado coinciden, ningún algoritmo puede hacerlo mejor.

Aclaro que eso es una cota de información, no una demostración de que greedy sea óptimo para cualquier catálogo. Con otro conjunto de personajes podría no serlo y habría que analizarlo de nuevo.

Cuántas preguntas necesita (la complejidad)

Cada turno recorro los candidatos una vez para partírlas, o sea $\Theta(n)$, y el conjunto se reduce a la mitad. La recurrencia es:

$$T(n) = T(n/2) + \Theta(n)$$

Es el caso de división con **a = 1, b = 2, k = 1**. Como $a < b^k$ ($1 < 2$), el costo de cómputo total queda en $\Theta(n)$.

Pero al juego lo que le importa no es el costo de cómputo sino **cuántas preguntas hago**, y eso es la profundidad de la recursión: **$\Theta(\log n)$** .

Forma de jugar	Peor caso	Orden
Ir probando personaje por personaje	23 intentos	$O(n)$

Forma de jugar	Peor caso	Orden
Partir el conjunto a la mitad	5 preguntas	$O(\log n)$

Esa diferencia entre 23 y 5 es toda la razón de ser del trabajo.

El catálogo: una cuenta que cambió todo

Los filtros que pide el enunciado son género (2 valores), calvicie (2), lentes (2) y color de pelo (3). Eso da:

$$2 \times 2 \times 2 \times 3 = 24 \text{ combinaciones posibles}$$

Y el enunciado pide **23 personajes**. O sea que entran justo, uno por combinación, y sobra una.

Aproveché eso: **mis 23 personajes usan 23 combinaciones distintas, ninguna repetida**. La consecuencia es que la máquina siempre termina con un único candidato y nunca tiene que adivinar al azar. El programa lo verifica solo al arrancar y tira una excepción si alguna vez se cargara un duplicado.

Para que la cuenta cierre tuve que tomar una decisión: **los personajes calvos también tienen color de pelo asignado** (se interpreta como el color de las cejas, las patillas o la barba). Si “calvo” anulara el color, las combinaciones bajarían de 24 a 16, y con 23 personajes forzosamente habría repetidos. Ahí la máquina llegaría a un empate de dos o tres personajes idénticos sin ninguna pregunta capaz de separarlos, y tendría que tirar una moneda. Todo el análisis algorítmico se caería justo al final.

Por qué son 6 filtros y no 9

Para género, calvicie y lentes hay **un solo filtro** por característica. Preguntar “¿es mujer?” cuando ya existe “¿es hombre?” no aporta nada: es la misma pregunta con la respuesta dada vuelta. Los tres colores sí van separados, porque son tres valores y no dos.

La carga de personajes y por qué no usé MergeSort

El enunciado pide que los personajes empiecen ordenados solo por género y que sea la máquina la que los disponga en una lista ordenada de forma autoincremental, a medida que se agregan.

Lo resolví así: cada personaje que entra recibe un **id incremental** (1, 2, 3...) y se **inserta en la posición que le corresponde** dentro de una lista que se mantiene siempre ordenada por atributos. Para encontrar esa posición uso búsqueda binaria recursiva, que es Divide & Conquer otra vez, aplicado en un segundo lugar del proyecto.

La recurrencia acá es $T(n) = T(n/2) + c$, o sea $a=1$, $b=2$, $k=0$. Como $a = b^k$ ($1 = 2^0$), queda $\Theta(\log n)$.

Lo bueno es que lo pude comprobar en mi propio programa. Esta es la cantidad de comparaciones que necesitó cada alta:

Comparaciones	Cuántos personajes
0	1
1	1
2	2

Comparaciones	Cuántos personajes
3	4
4	10
5	5

El primero necesita 0 y el vigésimo tercero necesita 5. Como $\log_2(23) \approx 4,5$, el máximo de 5 es exactamente lo que se espera. Si fuera una búsqueda lineal, el último habría necesitado hasta 22 comparaciones.

Por qué no MergeSort acá

Porque **MergeSort necesita el arreglo completo** para poder partirlo a la mitad, y en mi caso los personajes llegan de a uno. Tendría que reordenar toda la lista con cada alta: $\Theta(n \log n)$ por personaje.

La inserción binaria cuesta $\Theta(\log n)$ para encontrar la posición más $\Theta(n)$ para correr los que están detrás. Con $n = 23$ son unas 500 operaciones en total, o sea nada.

Quiero aclarar que **no es que MergeSort sea peor**: es que no corresponde a esta situación. Si el enunciado me hubiera pedido ordenar un lote completo de una vez, MergeSort sería la opción correcta.

Una prueba que hice y me sorprendió

Como no quería afirmar “greedy es mejor” sin poder demostrarlo, programé una simulación que juega **las 23 partidas posibles** (una por cada personaje secreto) con tres estrategias distintas:

Greedy completo: selección minimax + factibilidad.

Secuencial con factibilidad: pregunta en un orden fijo, pero saltea las preguntas cuya respuesta ya se conoce.

Secuencial pura: orden fijo y nada más.

Estrategia	Peor caso	Promedio	Aciertos
Greedy completo	6 turnos	5,61	23/23
Secuencial con factibilidad	6 turnos	5,61	23/23
Secuencial pura	7 turnos	6,96	23/23

El resultado no fue el que esperaba. Separando qué aporta cada elemento del greedy:

Elemento	Cuánto mejora el promedio
La función de factibilidad sola	-1,35 turnos
La función de selección minimax	0,00 turnos

O sea que en mi juego **lo que realmente mejora el rendimiento no es elegir la mejor pregunta, sino descartar las preguntas cuya respuesta ya está determinada**. Se ve clarísimo en esta salida del modo máquina contra máquina:

```
--- TURNO 6 - juega SECUENCIAL (8 candidatos) ---
```

```
[SECUENCIAL] SELECCION secuencial: "¿Tiene el pelo amarillo?", el primero  
pendiente del orden fijo. No evaluo cuanto corta.  
[SECUENCIAL] respuesta NO -> Candidatos 8 -> 8 (descarte 0%)
```

Gastó un turno entero sin descartar a nadie, porque los 8 candidatos que le quedaban eran todos de pelo amarillo y ya lo sabía.

¿Y por qué la selección minimax no aporta nada? Porque **mi catálogo es uniforme**: como usé una combinación de cada, los tres filtros binarios cortan 12/11 o 11/12. Están empatados. Greedy no puede sacar ventaja cuando no hay una mejor opción para descubrir.

Igual la dejé, por dos motivos: garantiza que nunca elija un filtro malo (los de color cortan 8/15 y 7/16), y con un catálogo desbalanceado sí haría diferencia. Lo que la medición me mostró es que **la ventaja de un algoritmo voraz depende de cómo estén armados los datos, no solo del algoritmo**.

Algoritmos que no apliqué

Esta parte la escribo porque el enunciado avisa que me pueden preguntar por los algoritmos **no** aplicados.

MergeSort y QuickSort. Ya lo expliqué arriba: ninguno de los dos sirve para inserción de a uno, porque necesitan el conjunto completo. Además QuickSort tiene peor caso $\Theta(n^2)$ cuando el pivot cae en un extremo, y eso pasa justamente con vectores ya parcialmente ordenados, que es mi caso porque los personajes llegan agrupados por género.

Programación dinámica. No aplica. Sirve cuando los subproblemas se repiten, como en el Fibonacci recursivo ingenuo que recalcula lo mismo millones de veces. En mi juego cada respuesta parte el conjunto en dos grupos disjuntos: nunca vuelvo a visitar el mismo subconjunto de candidatos, así que no hay nada que guardar. Memorizar resultados solo gastaría memoria sin ahorrar un solo cálculo.

Búsqueda exhaustiva del mejor orden de preguntas. Sería probar los $6! = 720$ órdenes posibles de filtros para encontrar la secuencia perfecta. Es computacionalmente posible, pero no aporta nada: greedy ya llega al mínimo teórico de 5 preguntas. No hay una secuencia mejor para encontrar.

Divide & Conquer para elegir el filtro. Lo pensé: partir la lista de filtros a la mitad, buscar el mínimo en cada mitad y combinar. Lo descarté porque encontrar el mínimo de una lista obliga a mirar todos los elementos igual, así que D&C no baja el orden (sigue siendo $\Theta(f)$) y solo agrega llamadas a la pila. El Divide & Conquer de mi proyecto está donde sí baja el orden: en la reducción de candidatos, de $\Theta(n)$ a $\Theta(\log n)$.

Otras decisiones de diseño

La máquina no puede espiar mi personaje. El enunciado lo pide explícitamente. Lo resolví con una interfaz llamada Oraculo que expone únicamente dos operaciones: preguntar por un filtro y arriesgar un nombre. El personaje secreto vive en un campo privado sin ningún getter. La máquina recibe la interfaz, no el objeto. No es que "no lo hace": no compilaría si lo intentara.

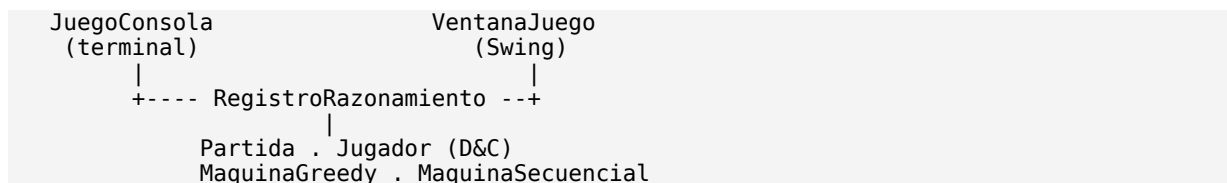
Dónde sí uso azar. Hay un solo Random en todo el proyecto, y es para que la máquina elija su propio personaje secreto. Eso no contradice la regla de que las decisiones de juego tengan un criterio explicable, porque elegir el propio personaje no es una decisión de juego, es la condición inicial. Al contrario: si la máquina eligiera siempre el primero de la lista, el rival lo sabría y ganaría en un turno.

La máquina de contraste tampoco juega al azar. MaquinaSecuencial pregunta en un orden fijo declarado de antemano. Juega peor a propósito, pero su criterio se explica en una frase: recorre los filtros en el orden en que fueron declarados y pregunta el primero que sea factible. Es una búsqueda ingenua, que es un concepto de la materia. No hay ningún Random en su lógica de decisión.

Si arriesgás y errás, perdés. Es la regla clásica y le da sentido a la decisión de arriesgar. Si equivocarse no costara nada, lo óptimo sería adivinar en el turno 1. Con esta regla, mi máquina greedy solo arriesga cuando le queda un único candidato, donde la probabilidad de acertar es 1.

Cómo está armado el proyecto

Hice dos interfaces que comparten el mismo motor:



La clave es que **el motor nunca llama a System.out**. Le pasa los mensajes a un RegistroRazonamiento, y hay tres implementaciones: RegistroConsola imprime, RegistroSwing escribe en la ventana, RegistroSilencioso descarta (lo uso para correr las 23 simulaciones sin llenar la pantalla).

Por eso, si se cierra la consola el juego sigue funcionando en Swing, y al revés. Ninguna clase del motor cambia una sola línea.

Otra decisión que me parece importante: **Partida no tiene un bucle adentro**. Expone “¿a quién le toca?”, “aplicá esta jugada” y “¿terminó?”, y la vista decide cuándo pedir la próxima jugada. En consola la llamo desde un while; en Swing, desde el clic de un botón. Si el bucle estuviera dentro del motor, Swing se congelaría esperando input.

Sobre la interfaz gráfica. El formulario lo diseñé con el Swing UI Designer de IntelliJ y quedó guardado en VentanaJuego.form. Los componentes se instancian por código en vez de usar el método que genera el diseñador, porque ese método depende de forms_rt.jar, una librería interna de IntelliJ, y eso haría que el proyecto no compile fuera del IDE. No se pierde nada, porque el layout definitivo lo arma reorganizarLayout() con BorderLayout y JScrollPane: la grilla que generaba el diseñador se descartaba igual, y además dejaba los botones fuera de la pantalla cuando el tablero crecía.

La ventana tiene un desplegable con los mismos cuatro modos que el menú de consola. Para el modo máquina contra máquina uso un **javax.swing.Timer** que muestra un turno por segundo, y no **Thread.sleep**: el hilo de Swing es el mismo que redibuja la pantalla, así que dormirlo congelaría la ventana. El Timer dispara sus eventos en ese hilo sin bloquearlo, entonces entre turno y turno la interfaz se sigue actualizando. Es el mismo motivo por el que Partida no tiene un bucle adentro.

Los modos de simulación y verificación redirigen System.out a un buffer temporal, corren la clase correspondiente y vuelcan el resultado en el panel de razonamiento. Así la ventana muestra exactamente la misma salida que la consola sin duplicar una sola línea de código.

Cómo ejecutarlo

Clase	Qué hace
JuegoConsola	Menú con los cuatro modos de consola
VentanaJuego	La interfaz gráfica
SimulacionEstrategias	Las 23 partidas con cada estrategia y la comparación
VerificacionCatalogo	La carga, la traza de inserción binaria y la verificación de unicidad

Los cuatro modos se pueden ejecutar tanto desde JuegoConsola como desde el desplegable de la ventana. El modo **máquina contra máquina** es el que pide el enunciado cuando dice que se tienen que poder presenciar todos los procesos que hace la máquina para acortar la búsqueda: cada turno muestra qué filtros evaluó, cuánto cortaba cada uno, cuál eligió, por qué, y en cuánto quedó el conjunto.

Resumen de complejidades

Operación	Complejidad
Partir el conjunto de candidatos	$\Theta(n)$
Evaluar todos los filtros de un turno	$\Theta(f \cdot n)$
Cantidad de preguntas de una partida	$\Theta(\log n)$
Encontrar la posición de un alta	$\Theta(\log n)$
Insertar un alta (con desplazamiento)	$\Theta(n)$
Carga completa del catálogo	$\Theta(n^2)$

Con 23 personajes y 6 filtros todos estos costos son insignificantes en tiempo real. El valor del análisis no está en el ahorro concreto, sino en entender que la diferencia entre $O(n)$ y $O(\log n)$ es la que separa 23 intentos de 5.